

a) Introduction:

The primary goal of this course is to give an insight of design & analysis of the algorithms. Throughout the semester, we have learnt some traditional algorithms. For instance, searching algorithms, sorting algorithms, algorithms to find the shortest path and so on. While practicing, implementing these, we frequently used a term called “Time complexity”. This name refers to the behaviour of the certain algorithm with respect to the input size. However, that asymptotic analysis of any algorithm was fully theoretical, so we were heading towards something that would connect the theory with the practical scenario and gave us the opportunity to visualize the behaviour of any algorithm using our own system. This project bridges the gap between our conceptual understanding and applied reasoning.

b) Algorithm Analysis:**1. Linear Search→****Pseudocode:**

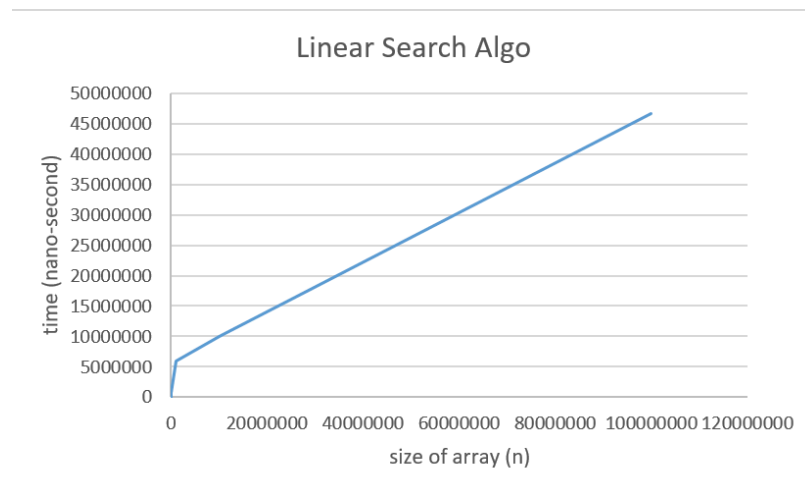
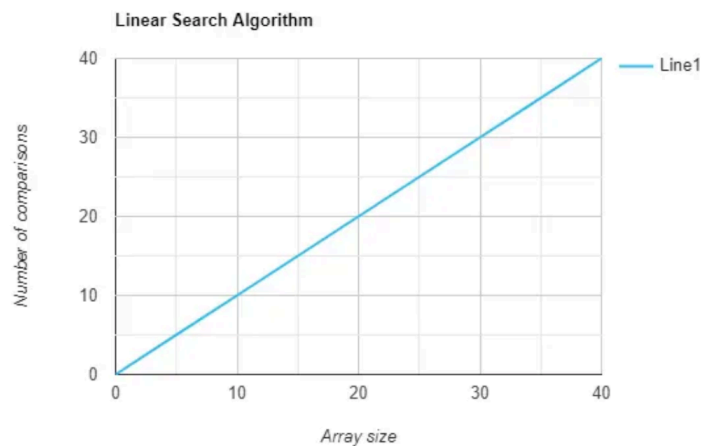
```
LinearSearch(array, key)
  for each item in the array //O(n)
    if item == value
      return its index //O(1)
```

Time Complexity: The worst case scenario of this linear search algorithm is $O(n)$. As it checks all the elements to match with the desired one.

Space Complexity: Since the algorithm does not consume any auxiliary space rather than the constant spaces of item, value and other variables, the space complexity of the linear search algorithm is $O(1)$.

Table & Diagram:

n (array size)	Time (nano-seconds)					
1	11700	7500	12000	10900	12300	avg=10880
10	9700	6400	10000	9700	6200	avg=8400
100	12300	8600	8800	13700	9400	avg=10560
1000	26200	22000	27600	35900	35700	avg=29480
10000	283000	281800	287200	282800	282600	avg=283480
100000	851600	402000	511600	836800	837100	avg=687820
1000000	6512100	6524600	4464200	6286500	5616600	avg=5880800
10000000	9028400	11931300	9582600	11722300	7275800	avg=9908080
100000000	48848900	50923000	45602900	42951000	44874100	avg=46639980

Experimental Analysis:Theoretical Analysis:

These two diagrams reflect that, our experimented times with respect to the input size is quite valid to visualize the time complexity of the Linear search algorithm.

2. Binary Search→

Pseudocode:

```

Sort the main array.
do until the pointers low and high meet each other.
    mid = (low + high)/2
    if (x == arr[mid])
        return mid
    else if (x > arr[mid]) // x is on the right side
        low = mid + 1
    else // x is on the left side
        high = mid - 1

```

Time Complexity: Each time while searching for the desired value x, the array becomes half and then finds the value in the right and left portion of the mid value. And the process continues until the value is found. So, if the size of the array is n–

$$\text{1st turn, } \frac{n}{2}$$

$$\text{2nd turn, } \frac{n}{2^2}$$

$$\text{3rd turn, } \frac{n}{2^3}$$

$$\text{When the desired value found then, } \frac{n}{2^k} = 1 \Rightarrow n = 2^k \Rightarrow k = \log_2 n$$

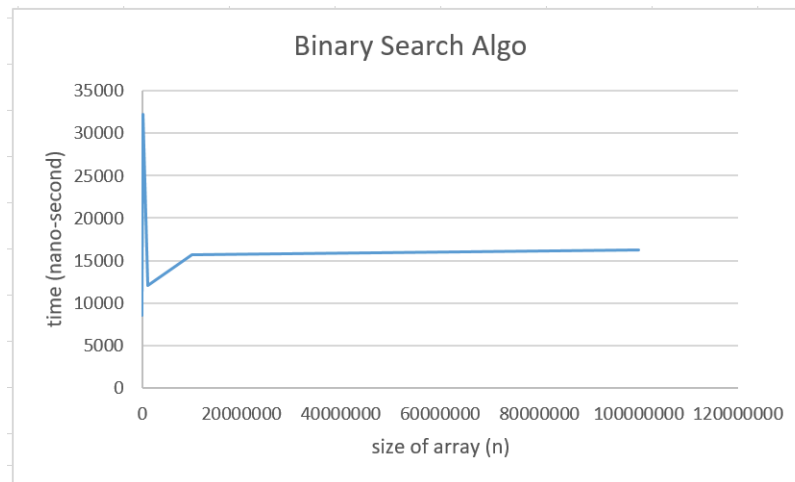
The time complexity is $O(\log n)$

Space Complexity: $O(1)$ is the space complexity of this algorithm. Because, no extra space is needed rather than the x,mid,high variables constant spaces. Moreover, the array is also used as an input ,so it will not consume auxiliary space.

This binary search algorithm is showing this behaviour just because each time the array gets half of its previous. The main backbone of this algorithm is the “Divide & Conquer” approach.

Table & Diagram:

n (array size)	Time (nano-seconds)					
1	14300	9400	13700	8800	9800	avg=11200
10	10600	7100	16800	9200	10200	avg=10780
100	6700	6500	11500	11200	7200	avg=8620
1000	12200	11400	11700	11700	11500	avg=11700
10000	11400	12200	12600	7400	13200	avg=11360
100000	31800	32700	32200	31200	33000	avg=32180
1000000	10800	14000	8700	13800	13100	avg=12080
10000000	16100	15200	15200	15200	17000	avg=15740
100000000	19300	11000	17400	15700	17700	avg=16220

Experimental Analysis:

3. Merge Sort →

Pseudocode :

MERGE_SORT(A):

 If $\text{length}(A)/2 > 1$:

$\text{mid} = \text{length}(A) / 2$

$\text{left} = A[0 : \text{mid}]$

$\text{right} = A[\text{mid} : \text{end}]$

 MERGE_SORT(left)

 MERGE_SORT(right)

 MERGE(A , left , right)

MERGE(A , left ,right):

$i = 0$

$j = 0$

$k = 0$

 while $i < \text{length}(\text{left})$ AND $j < \text{length}(\text{right})$:

 if $\text{left}[i] \leq \text{right}[j]$

$A[k] = \text{left}[i]$

$i = i + 1$

 else:

$A[k] = \text{right}[j]$

$j = j + 1$

$k = k + 1$

 while $i < \text{length}(\text{left})$:

$A[k] = \text{left}[i]$

$i = i + 1$

$k = k + 1$

 while $j < \text{length}(\text{right})$:

$A[k] = \text{right}[j]$

$j = j + 1$

$k = k + 1$

Time Complexity :

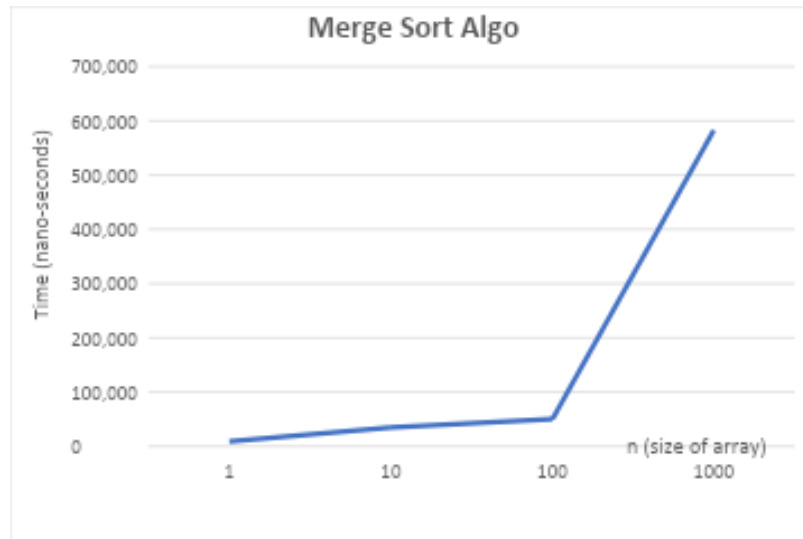
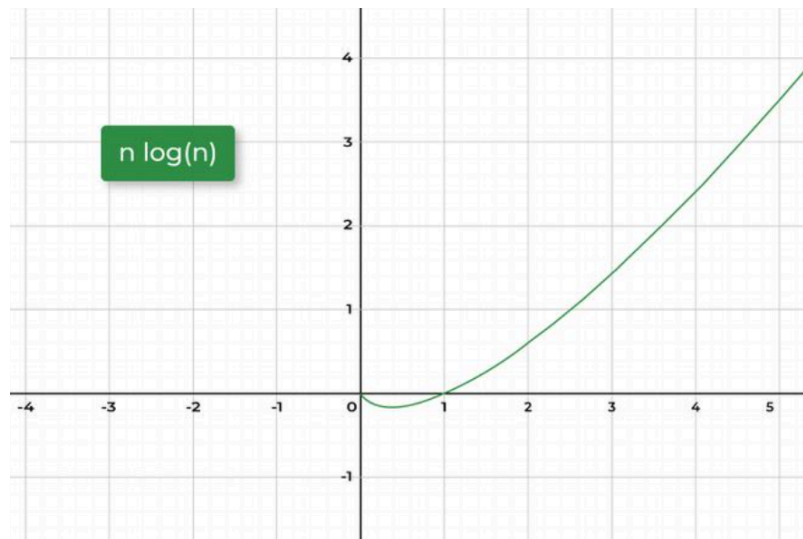
The merge sort follows divide, conquer and combine steps. First dividing the array into two arrays (**left = A[0 : mid]** , **right = A[mid : end]**), it takes $O(1)$ time. After that, it is recursively sorting both halves in **MERGE_SORT(left)**, **MERGE_SORT(right)** , for that merge sort is repeatedly divided by two. So, $n/2^k = 1$. After mathematical calculation the time complexity is $\log(n)$. The merge function touches each element once. So the work is being held n times, time complexity for that is $O(n)$. For total $\log(n)$ level, the work is being done n times, So total time complexity = $O(n \log n)$.

Space Complexity :

For space it creates a temporary array to store left and right and then merge. For that it takes no space. And for the recursion stack space it takes $\log(n)$ because it is splitting arrays each time. So, the space complexity is , $O(n) + O(\log n) = O(n)$.

Table & Diagram:

n (size of array)	Time (nano-seconds)					
1	9100	8400	9100	8200	8000	avg = 8,560
10	13300	49200	49900	12400	48800	avg = 34,720
100	49400	48200	50300	48300	52700	avg = 49,780
1000	591600	693800	550600	545100	533700	avg = 582,960

Experimental Analysis:Theoretical Analysis:

The second picture is showing the worst case time complexity of merge sort. Though we also tried to find the practical one, it bends when the input size is 100. We can say the both diagrams are pretty similar but it would be more precise if we could collect more datas for the graph.

4. Quick Sort →**Pseudocode :**

```

QUICKSORT(A, low, high):
    If low < high:
        pivotIndex = PARTITION(A, low, high)
        QUICKSORT(A, low, pivotIndex - 1)
        QUICKSORT(A, pivotIndex + 1, high)

PARTITION(A, low, high):
    pivot = A[high]
    i = low - 1

    for j = low to high - 1:
        if A[j] <= pivot:
            i = i + 1
            swap A[i] and A[j]

    swap A[i + 1] and A[high]
    return i + 1

```

Time Complexity :

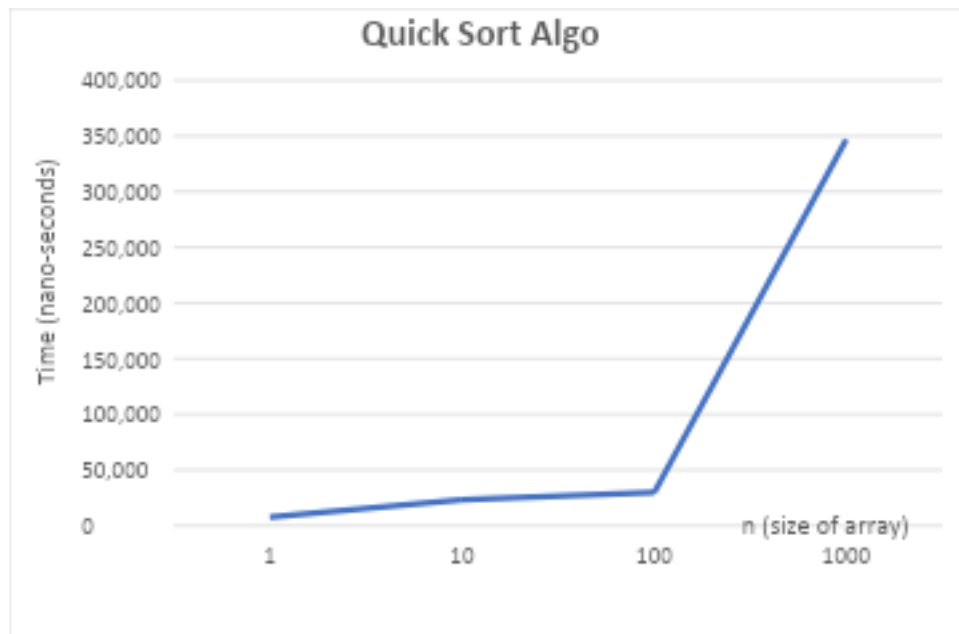
Time taken for partition is $O(n)$, because the loop (**for j = low to high -1:**) is running high to low times, which is n times. After running `PARTITION()`, `QUICKSORT()` recursively call two subarrays, which splits the array into two equal halves. So, $n/2^k = 1$. After mathematical calculation the time complexity is $\log(n)$. For total $\log(n)$ level, the work is being done n times, So total time complexity = $O(n \log n)$.

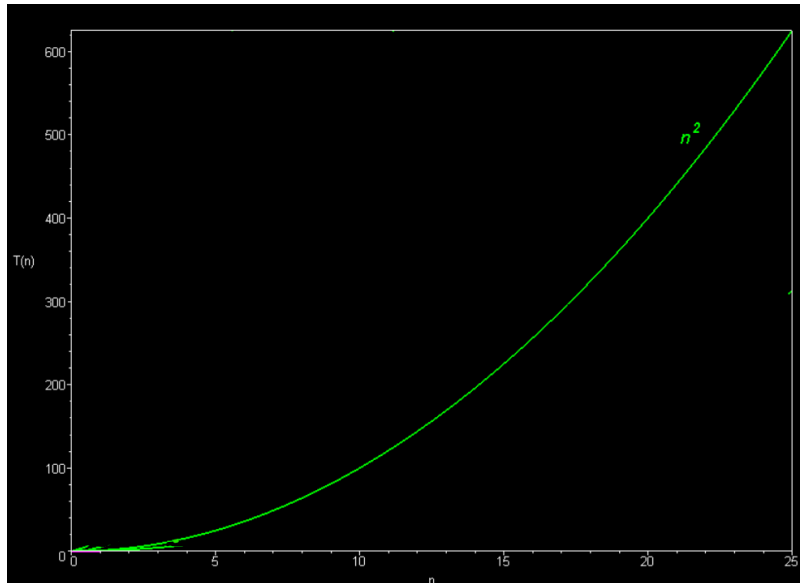
Space Complexity :

Quicksort is recursive, so its memory usage is from function call stack frames. Each recursive call uses space for variables low, high, i, pivot. The constant space for each call $O(1)$. But the space depends on the total recursive call recursion stack space it takes $\log(n)$. it's because of the equal split of each time. So, the space complexity of quick sort in the best case is $O(\log n)$, while in the worst-case scenario, it becomes $O(n)$ due to unbalanced partitioning causing a skewed recursion tree that requires a call stack of size $O(n)$.

Table & Diagram:

n (size of array)	Time (nano-seconds)					
1	8400	8200	7500	8200	6700	avg = 7,800
10	22800	23200	23100	22900	22900	avg = 22,980
100	35100	34500	34600	8700	34900	avg = 29,560
1000	339700	339100	345000	358200	351700	avg = 346,740

Experimental Analysis:

Theoretical Analysis:**5. Recursive Fibonacci→****Pseudocode:**

Step-1: Define the base cases.

If $n == 0$ return 0. If $n == 1$ return 1.

Step-2: For a general $n > 1$, make two recursive calls.

Call $\text{fib}(n-1)$ and $\text{fib}(n-2)$ to compute the two previous fibonacci numbers.

Step-3: Combine results of recursive calls.

Add the two return values: $\text{result} = \text{fib}(n-1) + \text{fib}(n-2)$.

Step-4: Return the combined result.

Return result to the caller.

Step-5: Each call returns its result up the call stack until the original call receives the final answer.

Time Complexity:

In the recursive Fibonacci algorithm, each call to $\text{fib}(n)$ makes two more calls $\text{fib}(n-1)$ and $\text{fib}(n-2)$. So the total number of calls creates a binary tree. As n increases, the number of nodes (function calls) in this tree approximately doubles at each level. This creates exponential growth, So, Time Complexity of $O(2^n)$.

Space Complexity:

The space used comes from the call stack depth during recursion.

At most, the function keeps calling $\text{fib}(n-1)$, then $\text{fib}(n-2)$, creating a chain down to the base case. This chain can be as long as n calls.

Therefore, the maximum stack depth is n , giving a space complexity of $O(n)$.

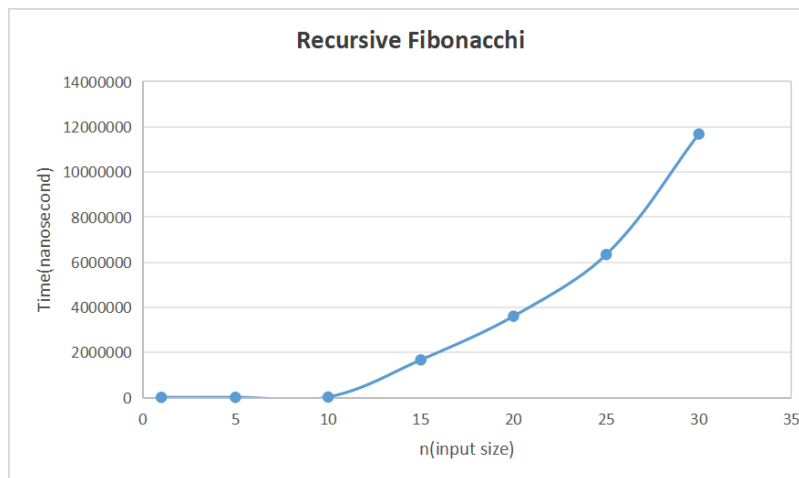
The recursive Fibonacci algorithm behaves this way $O(2^n)$ because each call generates two more calls ($\text{fib}(n-1)$ and $\text{fib}(n-2)$), creating an exponentially growing recursion tree where many values are recomputed repeatedly. Its space is $O(n)$ because the deepest path of recursive calls can go down to $\text{fib}(0)$ or $\text{fib}(1)$, requiring a call stack of size n .

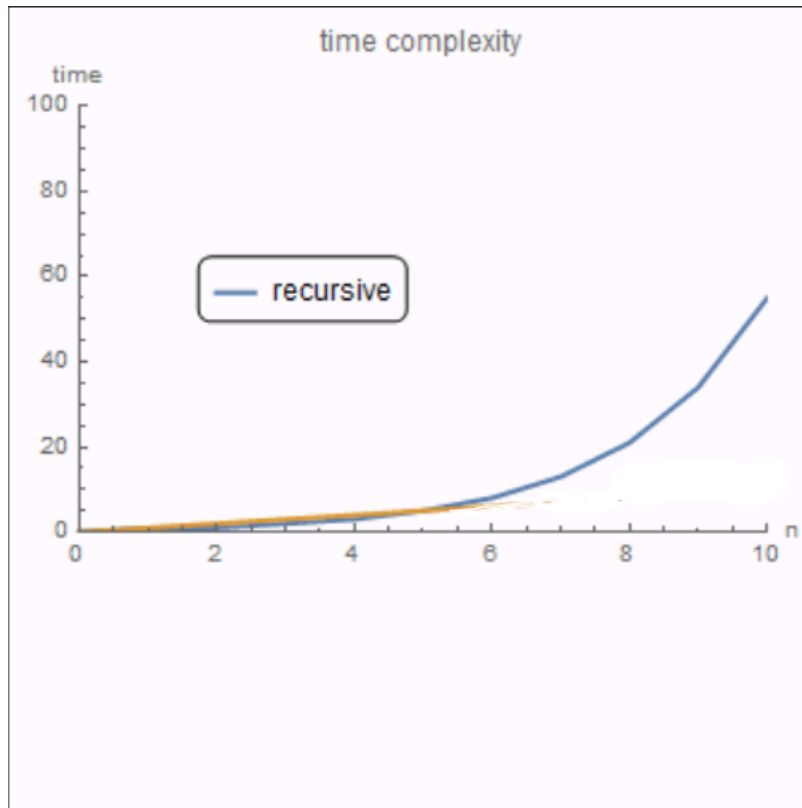
Table and Diagram:

n(input size)	Time(nanosecond)					
1	9500	8700	9400	9800	9000	avg=9280
5	11200	12400	11600	11400	10400	avg=11400
10	22700	23000	24800	22900	25500	avg=23780
15	5978100	609100	572000	614200	582400	avg=1671160
20	7936800	793500	7658100	794500	847700	avg=3606120
25	6796900	6190700	5145300	7077100	6486300	avg=6339260
30	15973700	12633900	16257600	15240700	12926500	avg=11680112

***** Why should n (input size) not be 1000 for Recursive Fibonacci algorithm?**

Answer: This happens because each call creates two more calls, creating a huge binary recursion tree that grows exponentially. As n increases, the number of function calls almost doubles at each step, (2^n), making the algorithm extremely slow for large n. That's why it is only practical to run for small inputs like $n \leq 30$, and impossible for very large values like $n = 1000$.

Experimental Analysis:

Theoretical Analysis:

After analyzing both diagrams, we can give a verdict that the recursive fibonacci perfectly behaves exponentially.

6. 0/1 Knapsack →**Pseudocode :**

```
Knapsack(W, wt[], val[], n):
    create 2D array dp of size (n+1) x (W+1)
    for i = 0 to n:
        dp[i][0] = 0
    for w = 0 to W:
        dp[0][w] = 0
    for i = 1 to n:
```

```

for w = 1 to W:
    dp[i][w] = dp[i-1][w]
    if wt[i] <= w:
        dp[i][w] = max(dp[i][w],
                        dp[i-1][w - wt[i]] + val[i])
return dp[n][W]

```

Time Complexity :

The 0/1 Knapsack algorithm uses Dynamic Programming, where we build a 2D Dynamic Programming table of size $(n + 1) \times (W + 1)$, where n = number of items, W = knapsack capacity. The outer loop runs for every item for n iteration. The inner loop runs for every weight capacity from 0 to W iterations. So, the time complexity for the loop is $O(nW)$. The constant time work which is, comparing two values, adding value of an item all work takes $O(1)$ time. So the total time complexity is $O(nW)$.

Space Complexity :

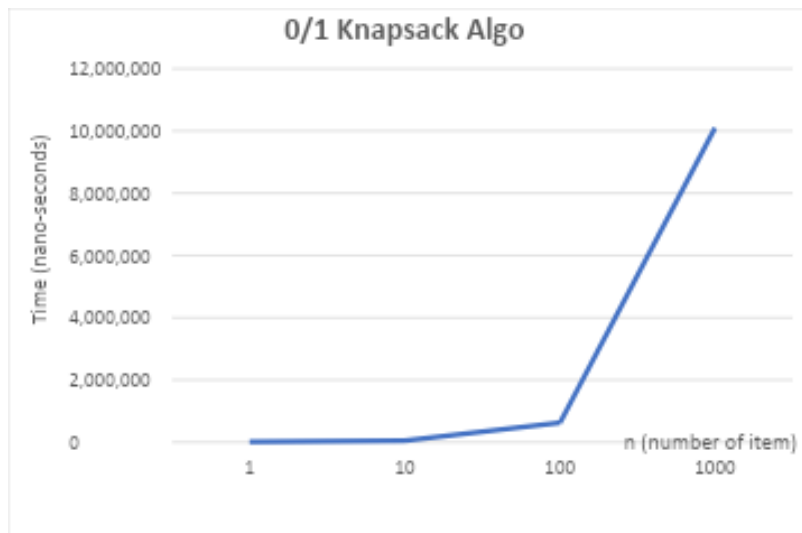
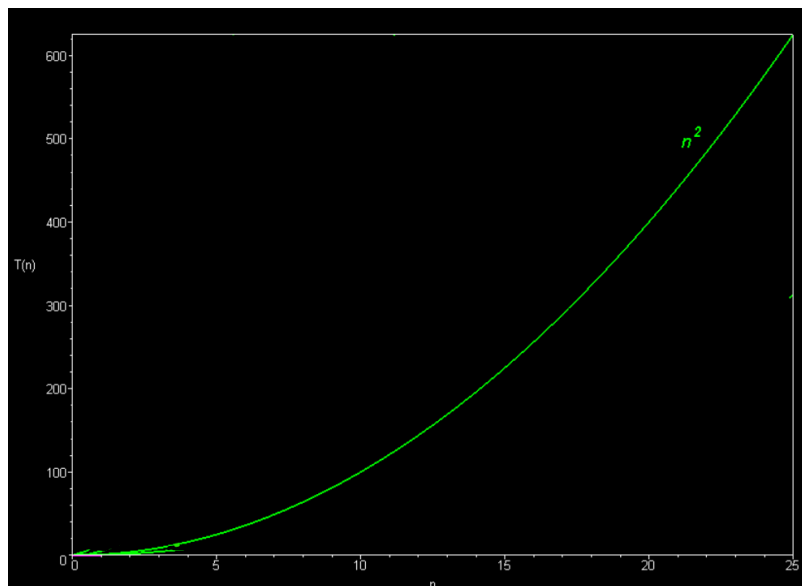
It has a few simple variables like i , w , temporary values, all taking $O(1)$ space. For space, the algorithm creates a 2D DP table with dimensions $(n + 1) \times (W + 1)$. This table stores the maximum value achievable for each item and capacity. So the memory used is $O(nW)$.

Table & Diagram:

n (Num of items)	Time (nano-seconds)					
1	11200	41400	9500	11200	9000	avg = 16,460
10	59100	40500	50500	10900	42400	avg = 40,680
100	498700	567900	659700	714900	671900	avg = 622,620
1000	10068400	9105500	9753400	10495900	11050000	avg = 10,094,640
10000	401858600	369706600	379012700	385399300	386299100	avg = 384,455,260

Note :

Here we have taken the Knapsack capacity as the number of items.

Experimental Analysis:Theoretical Analysis:

Both of the line graphs are somehow increasing at an increasing rate. However, due to the limitations of machines, the experimental outcome is not as smooth as it should be for a graph like n^2 . Point to be noted, 0/1 Knapsack is a NP-complete problem, so here we have just implemented its pseudo polynomial time version. That is why the time complexity is like that.

7. Breadth-First Search (BFS) →

Pseudocode:

Step-1: Find the in-degree (number of incoming edges) for every vertex in the graph.

Step-2: Create a queue and insert all vertices whose in-degree is 0.

Step-3: While the queue is not empty,

- Remove one vertex from the front of the queue.
- Add that vertex to the topological orderlist.
- For each vertex that the removed vertex point to:
 1. Reduce that neighbour's in-degree by 1.
 2. If the in-degree of that neighbour becomes zero, add it to the queue.

Step-4: Continue until the queue is empty.

Time Complexity:

V (The number of vertices)

E (The number of edges)

Each vertex is enqueued and dequeued exactly once. So, Cost $\rightarrow O(V)$.

For each vertex, we check all its edges. So, Cost $\rightarrow O(E)$.

So, the Total Time Complexity = $O(E+V)$

For a dense graph, $E = n^2$, so the time complexity becomes $O(n+n^2) = O(n^2)$

Space Complexity:

- ❖ For, Queue + Visited array = $O(V)$
- ❖ For, Graph + BFS storage = $O(V+E)$

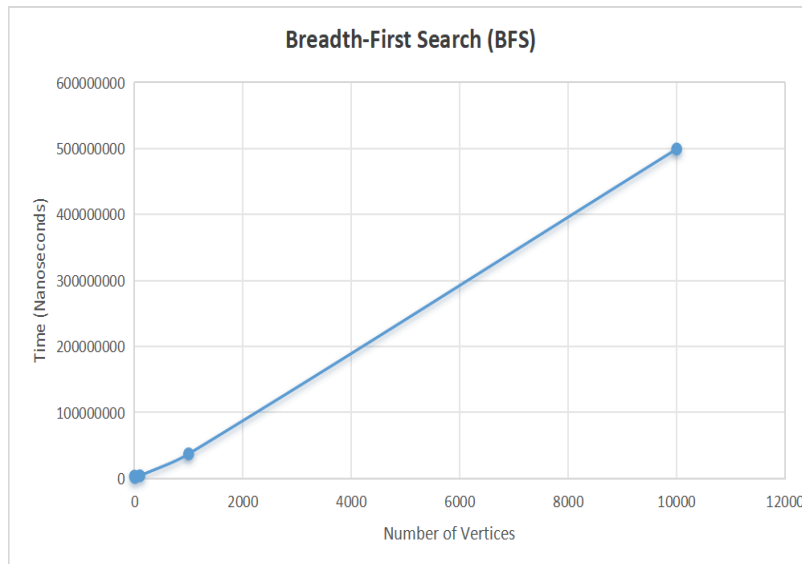
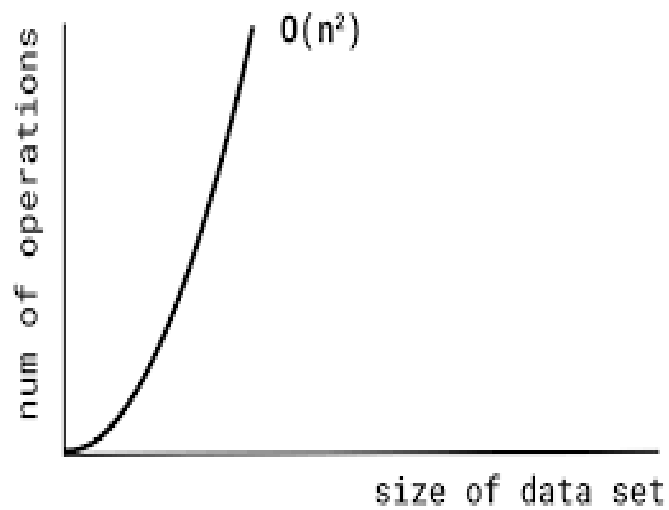
So, The Space Complexity = $O(V+E)$

For a dense graph, $E = n^2$, so the space complexity becomes $O(n+n^2) = O(n^2)$

BFS exhibits this behaviour because it visits all vertices and their neighbors using a queue. In a dense graph, each vertex is connected to almost every other vertex, so the algorithm processes roughly n^2 edges. This causes the runtime to grow quadratically with n .

Table and Diagram:

n(number of vertices)	Time(nano-second)					
1	5289300	2268000	5831400	852700	2927700	avg= 3433820
10	1683200	1043700	1172100	1281700	1584700	avg=1353080
100	3367700	3244400	6230800	3251000	3772600	avg=3973300
1000	36844600	36829700	39261300	36168300	36665100	avg=37153800
10000	564038900	481410600	497834600	505616000	447280500	avg=499236120

Experimental Analysis:Theoretical Analysis:

The reason the experimental graph looks linear is because of the plotting scale, CPU memory/cache limits, and small data range, which hides the curvature.

8. Dijkstra's Algorithm→**Pseudocode:**

function dijkstra (graph,source)

1. V = Number of Vertices
2. $\text{dist}[v] = \infty$ (Infinity)
3. $\text{visited}[v] = \text{false}$
4. $\text{dist}[\text{source}] = 0$
5. For $i = 0$ to V
 - $u = \text{vertex with minimum distance}[u]$
 - $\text{visited}[u] = \text{true}$
 - For $V = 0$ to V
 - If $\text{visited}[v] = \text{false}$ and
 - $\text{dist}[u] + w(u,v) < \text{dist}[v]$ then
 - $\text{dist}[v] = \text{dist}[u] + w(u,v)$
 - end if
 - end for
6. display dist
- end function

Time Complexity:

Outer loop runs $(k-1)$ times $\rightarrow O(V)$

Inner loop runs $(k-1)$ times $\rightarrow O(V)$

So, Total Time Complexity = $O(V^2)$

Space Complexity:

$\text{dist}[v]$ takes space of $O(V)$

$\text{Visited}[v]$ takes space of $O(V)$

$w[u][v] = O(V^2)$

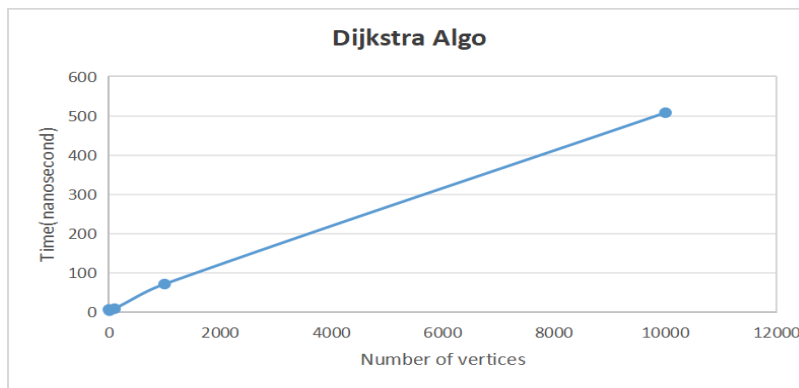
So, Total Space Complexity = $O(V^2)$

This Dijkstra algorithm behaves as $O(V^2)$ because in every iteration it linearly searches all vertices to find the smallest unvisited distance ($O(V)$) and then checks all neighbors in a dense graph ($O(V)$), and repeating this V times produces $O(V^2)$.

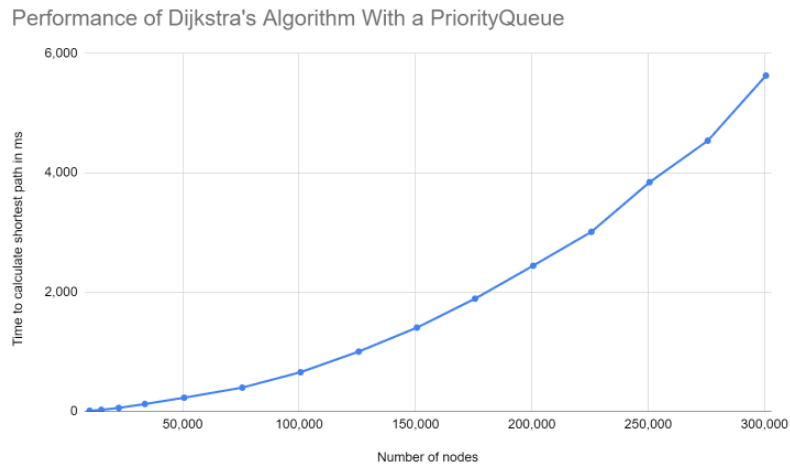
Table and Diagram:

n(Number of Vertices)	Time(nanosecond)					
1	7964400	6651600	5505300	6074700	6431300	avg= 6525460
10	3763700	3132800	4003500	1933700	3628900	avg= 3292520
100	7923800	8516900	6888200	9724700	8410100	avg= 8292740
1000	79640500	70188200	68169800	68510000	69143900	avg= 71130480
10000	520869900	502122300	494797600	505641900	515766600	avg= 507839660

Experimental Analysis:



Theoretical Analysis:



Here we have to say that, the 2nd figure named theoretical analysis is not a theoretical one! Rather, it is a different approach to find the shortest path from a single source. So obviously the graph will not be the same. However, our practical graph is not even visually quadratic as well, due to the limitations of the machine or the lack of input data.

9. Prim's algorithm→

Pseudocode:

Prim(G , start):

 for each vertex v in G : // $O(V)$

$\text{key}[v] = \infty$

$\text{parent}[v] = \text{NIL}$

$\text{key}[\text{start}] = 0$

$\text{PQ} = \text{priority queue containing all vertices by key}[]$

 while PQ not empty: // $O(V)$

$u = \text{extract-min}(\text{PQ})$ // $O(\log V)$ [priority queue]

 for each (u, v) with weight w : // $O(E)$

 if v in PQ and $w < \text{key}[v]$:

$\text{key}[v] = w$

$\text{parent}[v] = u$

$\text{decrease-key}(\text{PQ}, v, w)$ // $O(\log V)$ [priority queue]

 return parent

Time Complexity: The time complexity of the algorithm is $O(E \log V)$ [$O(V) + O(V) \times O(\log V) + O(E) \times O(\log V) = O(E \log V)$ as $E > V$]. The other non-mentioned statements have a complexity of $O(1)$ that are neglected in this scenario.

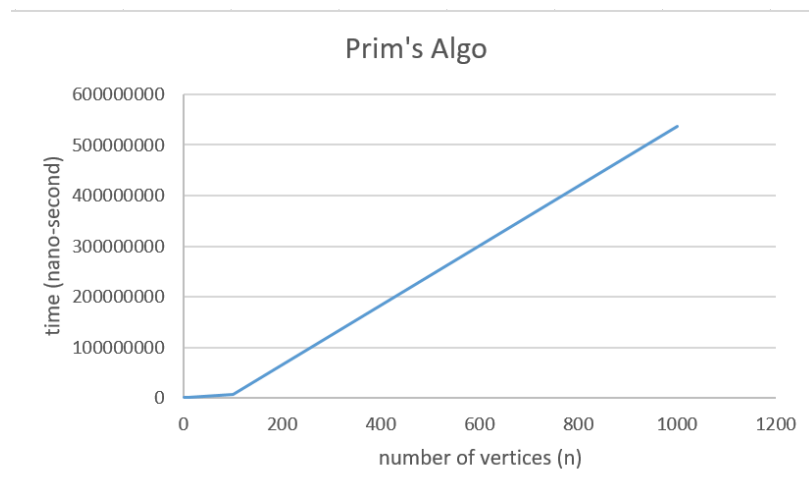
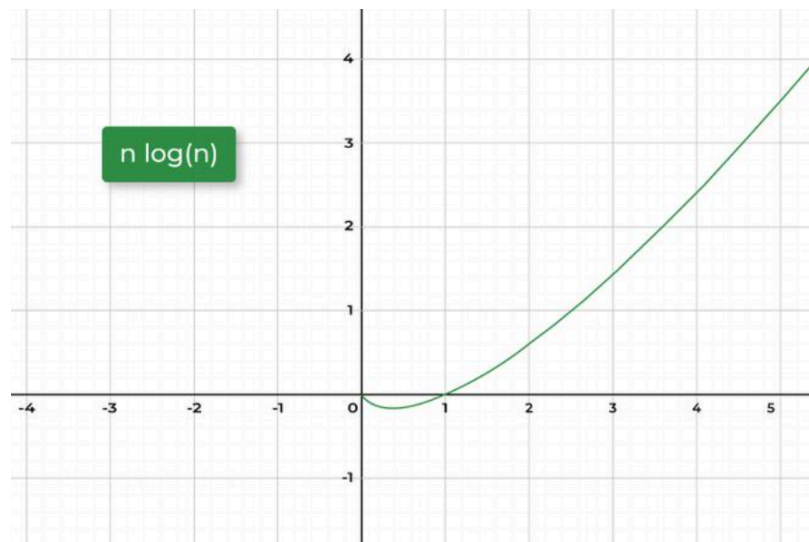
Here, the second loop iterates for V times and the last inner loop runs for E times.

Space Complexity: The space complexity of this algorithm is $O(V)$ because there is an array that is used to maintain the priority queue. Other arrays are also used of size V to keep the distances and parents track.

The algorithm behaves like this because of the usage of a priority queue. It uses the min heap sorting technique so that it shows $\log V$ complexity for each vertex. To sum up, the behaviour might be slightly changed based on the implementation.

Table & Diagram:

n (number of vertices)	Time (nano-seconds)					
1	502550	526080	729630	500151	422169	avg=536 116
10	539940	1038409	557210	701531	492529	avg=665 923.8
100	7113480	5544179	7146689	7676959	6795280	avg=685 5317.4
1000	6473281 90	4574934 75	5431898 02	4830808 54	5528750 62	avg=536 793476.6

Experimental Analysis:Theoretical Analysis:

10. Floyd-Warshall's algorithm→**Pseudocode:**

```

n = no of vertices
A = matrix of dimension n*n
for k = 1 to n // O(n)
    for i = 1 to n // O(n)
        for j = 1 to n // O(n)
             $A^k[i, j] = \min (A^{k-1}[i, j], A^{k-1}[i, k] + A^{k-1}[k, j])$ 
return A

```

Time Complexity: As there are three nested loops in the algorithm, the time complexity of this should be $O(n^3)$

$[O(n) \times O(n) \times O(n) = O(n^3)]$. The other non-mentioned statements have a complexity of $O(1)$ that are neglected in this scenario.

Space Complexity: The space complexity of this algorithm is $O(n^2)$ as there is a two dimensional array ($n \times n$) (2D-matrix) to store the shortest distance between two vertices. The other variables like “i,j,k” take constant spaces so the space complexity for those is $O(1)$, but this value can be neglected.

The Floyd-Warshall's algorithm exhibits this behaviour because it goes through each intermediate node (k) just to check whether there is any possible shortest path to go from i to j.

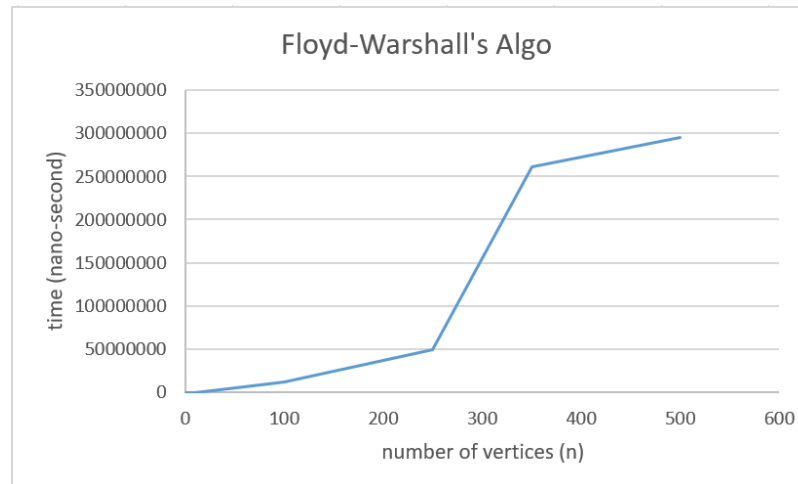
Table & Diagram:

n (number of vertices)	Time (nano-seconds)					
1	25550	16500	22940	19750	48720	avg=266 92
10	177520	153390	116240	116740	135330	avg=139 844

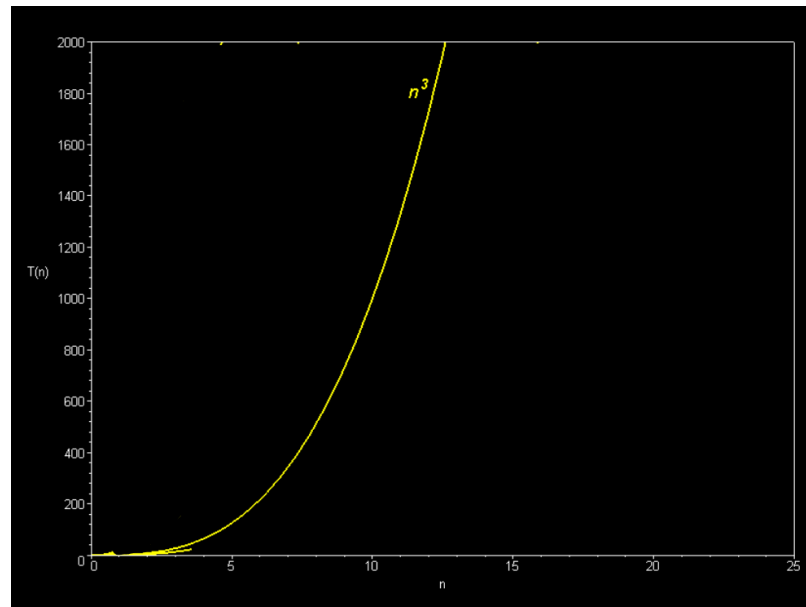
100	11481428	15393349	10586009	8902350	12248389	avg=11722305
250	52320846	38310347	51113296	60753096	44665287	avg=49432574.4
350	130792000	346772563	602249644	105895585	117225023	avg=260586963
500	269926182	295020488	329723703	274701781	304907877	avg=294856006.2

Though we tried to generate results for $n > 500$ as well but from that point, the behaviour of this algorithm becomes too slow. I ran for those, it took some minutes but still did not provide any output.

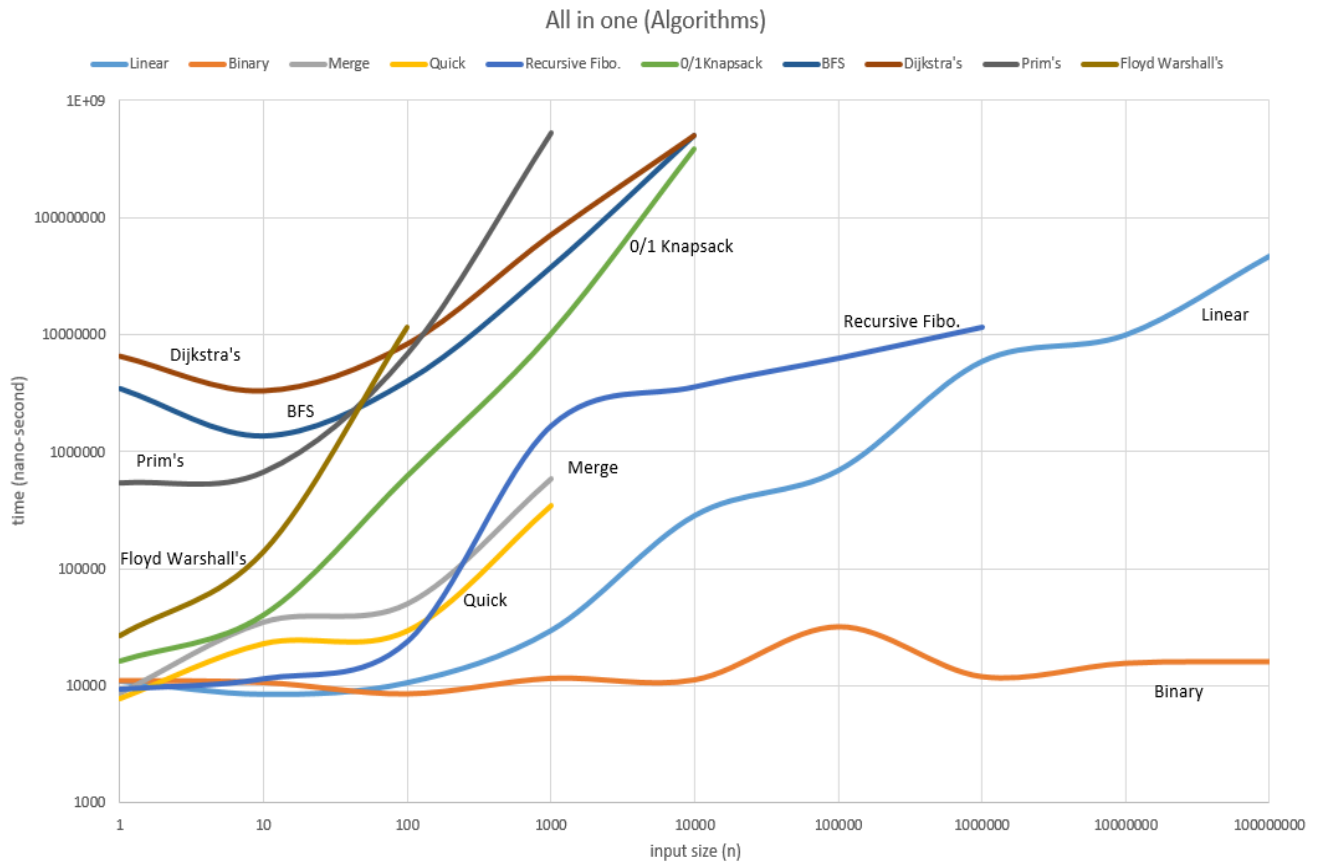
Experimental Analysis:



Theoretical Analysis:



c) Performance Graph:



In this final performance analysis, the axes are in logarithmic scale. Since, we cannot visualize and compare the growth of algorithms properly as there are times for very large inputs (10^8) and for also short inputs (100) .

d) Conclusion & Discussion:

To sum up, the empirical data generated by us slightly match the theoretical Big-O complexities. For better understanding, we have shown both theoretical and experimental diagrams in the algorithms section. However, the final performance graph is the output of all algorithms run times in logarithmic scale. So it will not be an ideal decision to compare the behaviour from here. In addition, these are generated just for short inputs, if the system was capable enough to deal with large datasets then the values would be more precise. Moreover, there will always be limitations, like in recursive fibonacci, we cannot run it for large values, as it may cause stack overflow at some point. For a while, if we consider $n=1$ then almost six algorithms took around 10000 nanoseconds to execute. Then the time fluctuates for the increasing size of input. This is how we can analyze the algorithms.

Special Problem

Travelling Salesman Problem (TSP):

The Travelling Salesman Problem (TSP) is a problem where the salesman has to visit all the cities exactly once and return to its starting city, just like a **Hamiltonian** cycle. There can be multiple Hamiltonian cycles in a TSP. The goal is to find that one Hamiltonian cycle with the minimum cost, meaning- minimum cost of travelling all cities exactly once and returning to its starting vertex. In this way the number of possible routes grows extremely fast as the input size (number of cities) increases. In **Brute Force** for n cities its complexity is $n!$. So, it is considered as the hardest problem in the Non-Polynomial (**NP-hard**) class. Because there is no known efficient algorithm that can solve it in a Polynomial time for large inputs. That is why the TSP problem is called an NP-hard problem. However, we can optimize it using **Dynamic Programming (Held Karp)** for which the complexity reduces to $n^2 * 2^n$. But it is still exponential and only practical for smaller values of n .

- **Pseudocode (Brute Force) :**

Step-1: Listing all cities to be visited.

Step-2: Finding all possible permutations of these cities.

Step-3: For each permutation, it calculates the total cost of the route (visit all cities and return to start) and keep track of the minimum cost found so far.

Step-4: After checking all permutations, the minimum cost found is the final result.

Step-5: Return the minimum cost.

Time Complexity:

The algorithm tries all possible permutations of visiting n cities.

There are $n!$ permutations and each permutation's cost is calculated once.

So, the total runtime grows factorially, $O(n!)$ time complexity.

Space complexity:

The algorithm stores the current permutation of cities in an array of size n .

No additional large data structures are needed.

So, the total space used is $O(n)$.

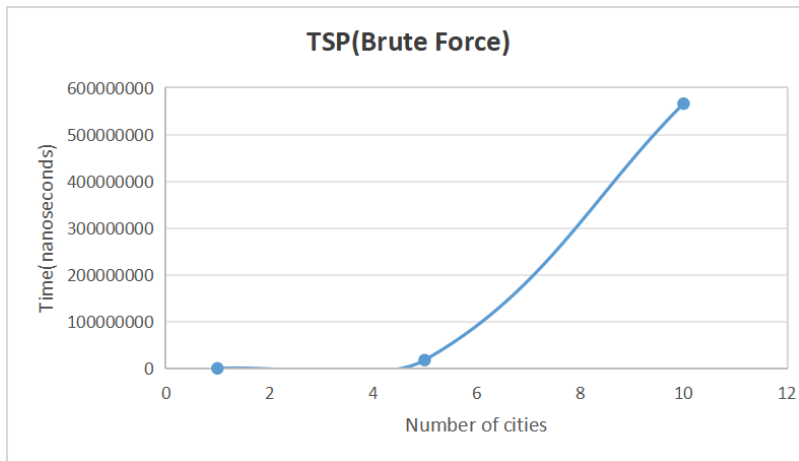
This algorithm behaves like $O(n!)$, because it checks every possible route of the cities to find the minimum cost. As the number of cities grows, the number of routes grows extremely fast, making it very slow for large n .

Table and Diagram:

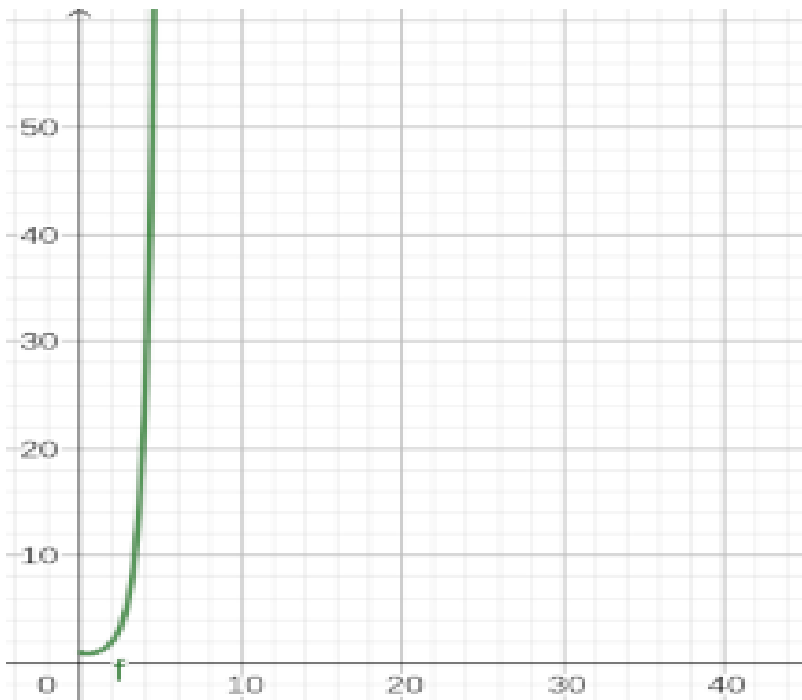
n(number of cities)	Time(nano-seconds)					
1	98100	65900	57500	283600	47000	Avg=110420
5	58319000	9053600	5347700	7216400	8413500	Avg=17670040
10	589502300	260467900	644397700	442823700	892499500	Avg=565938220

At $n > 11$, it becomes impossibly slow to run.

Experimental Analysis:



Theoretical Analysis: $n!$



TSP (Dynamic Programming-Held Karp):

- **Pseudocode (Held Karp):**

Step-1: Numbering all cities from 0 to $n-1$.

City 0 is the starting city.

Step-2: Create a DP table $dp[set][city]$,

that stores the minimum cost to start from city 0, visit all cities in set, and end at city.

Step-3: Initializing the base case.

Set $dp[\{0\}][0] = 0$ (only the start city is visited, cost is zero).

Step-4: For all possible sets of cities that include the start city:

1. For each city v in the set: Try coming to v from every other city u in the set ($u \neq v$).
2. Update $dp[set][v] = \min(dp[set - \{v\}][u] + cost[u][v])$.

Step-5: After all cities are visited, complete the tour.

Add the cost of returning from the last city back to the starting city.

Step-6: The minimum total cost found is the optimal TSP solution.

Time Complexity:

The algorithm looks at every possible group of cities, and for n cities there are 2^n such groups.

For each group, it checks all possible cities and paths between them, which takes about $n \times n$ operations.

Because this is done for all 2^n groups, the total running time becomes $O(n^2 \cdot 2^n)$.

Space Complexity:

The algorithm stores results for every subset of cities and every ending city in a DP table.

There are 2^n subsets and up to n cities for each subset.

So the total Space Complexity is $O(n \cdot 2^n)$.

This algorithm behaves this way because it checks all possible subsets of cities and stores their minimum travel costs. As the number of cities increases, the number of subsets grows exponentially, which makes the algorithm slow for large inputs.

Table and Diagram:

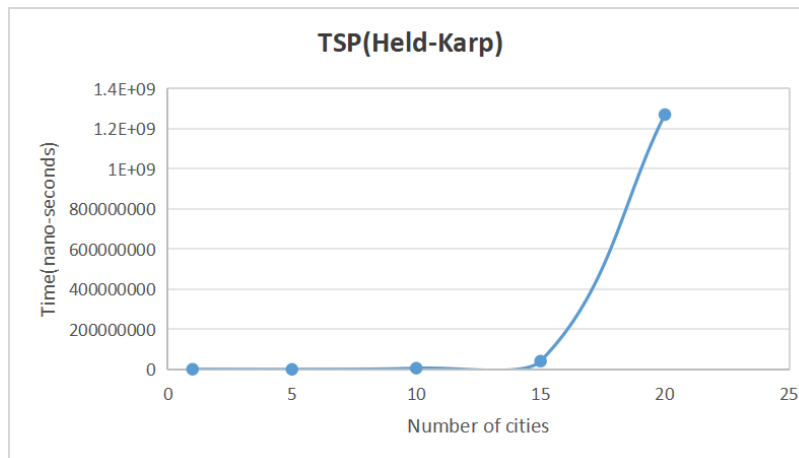
n(number of cities)	Time(nano-seconds)					
1	2363400	295600	43900	1025800	38600	Avg=753460
5	314300	211700	132200	137500	236100	Avg=206360
10	3860200	12669500	5274100	3397000	5126300	Avg=6065420
15	42076000	49714000	37128500	44154000	33514900	Avg=41317480
20	1932458100	1424338200	988939800	1698009600	1838452700	Avg=1270797952

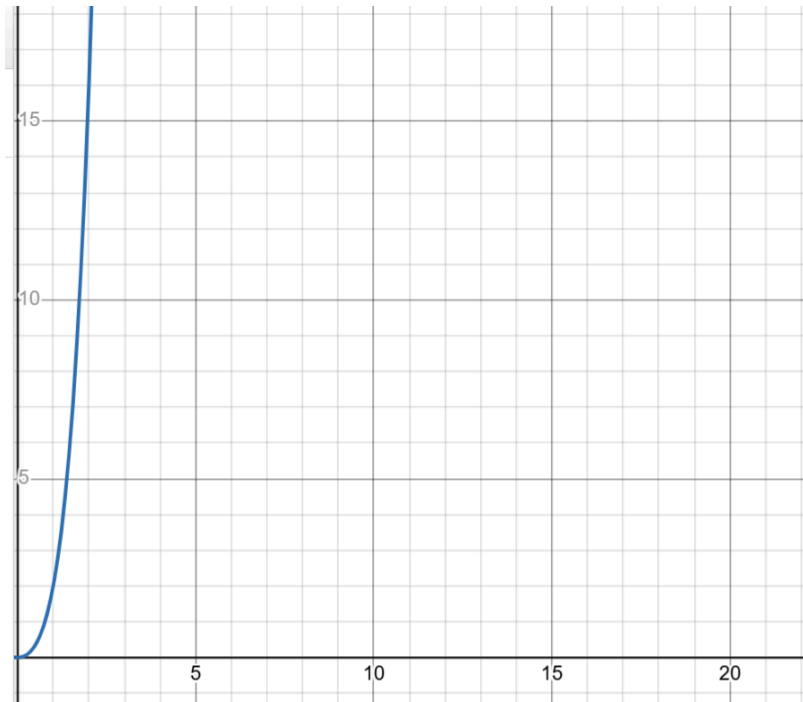
Held–Karp uses bitmask dynamic programming, which requires 2^n groups.

When n becomes large, memory and time grow exponentially and exceed computer limits.

That's why this algorithm is only practical for $n \leq 20$.

Experimental Analysis:



Theoretical Analysis: $n^2 2^n$ 

Theoretical graph assumes it is an ideal machine and ignores real world factors. That is why, it looks smooth and rises very fast. On the other hand, an experimental graph measures the actual running time by running the code on a real computer. It is affected by hardware speed, memory, compiler, and system load, so small inputs may look almost flat at first. As n increases, the exponential nature becomes visible, That is why the curve suddenly rises sharply in the experimental graph.

e) Appendix:

1) Linear Search:-

```
import java.util.*;

class GFG {

    // Linear Search
    public static int search(int arr[], int N, int x) {
        for (int i = 0; i < N; i++) {
            if (arr[i] == x)
                return i;
        }
        return -1;
    }

    public static void main(String args[]) {
        Scanner sc = new Scanner(System.in);

        // Take user input for array size
        System.out.print("Enter size of array (n): ");
        int n = sc.nextInt();

        // Generate array with random numbers 1 to n
        int arr[] = new int[n];
        Random rand = new Random();

        for (int i = 0; i < n; i++) {
            arr[i] = rand.nextInt(n) + 1; // values always in [1, n]
        }

        // Take the number to search
        System.out.print("Enter number to search: ");
        int x = sc.nextInt();

        // Measure time
        long start = System.nanoTime();
        int result = search(arr, n, x);
        long end = System.nanoTime();

        long duration = end - start;

        // Output
        if (result == -1)
            System.out.println("Element not found.");
        else
```

```

        System.out.println("Element found at index: " + result);

        System.out.println("Time taken (nanoseconds): " + duration);
    }
}

```

2) Binary Search:-

```

import java.util.*;

class GFG {

    // Binary Search
    static int binarySearch(int arr[], int x) {
        int low = 0, high = arr.length - 1;

        while (low <= high) {
            int mid = low + (high - low) / 2;

            if (arr[mid] == x)
                return mid;

            if (arr[mid] < x)
                low = mid + 1;
            else
                high = mid - 1;
        }
        return -1; // not found
    }

    public static void main(String args[]) {
        Scanner sc = new Scanner(System.in);

        // Input array size
        System.out.print("Enter size of array (n): ");
        int n = sc.nextInt();

        // Generate sorted random array from 1 to n
        int arr[] = new int[n];
        Random rand = new Random();

        for (int i = 0; i < n; i++)
            arr[i] = rand.nextInt(n) + 1; // ensure values always 1..n

        Arrays.sort(arr); // MUST sort for binary search
    }
}

```

```

// Input search number
System.out.print("Enter number to search: ");
int x = sc.nextInt();

// Measure time
long start = System.nanoTime();
int result = binarySearch(arr, x);
long end = System.nanoTime();

long duration = end - start;

// Output results
if (result == -1)
    System.out.println("Element NOT found.");
else
    System.out.println("Element found at index: " + result);

System.out.println("Time taken (nanoseconds): " + duration);
}
}

```

3) Merge_sort:-

```

import java.util.*;

public class MergeSortUser {

    public static void mergeSort(int[] arr, int left, int right) {
        if (left < right) {
            int mid = (left + right) / 2;
            mergeSort(arr, left, mid);
            mergeSort(arr, mid + 1, right);
            merge(arr, left, mid, right);
        }
    }

    public static void merge(int[] arr, int left, int mid, int right) {

```

```
int n1 = mid - left + 1;
int n2 = right - mid;

int[] L = new int[n1];
int[] R = new int[n2];

for (int i = 0; i < n1; i++) L[i] = arr[left + i];
for (int i = 0; i < n2; i++) R[i] = arr[mid + 1 + i];

int i = 0, j = 0, k = left;

while (i < n1 && j < n2) {
    if (L[i] <= R[j]) arr[k++] = L[i++];
    else arr[k++] = R[j++];
}

while (i < n1) arr[k++] = L[i++];
while (j < n2) arr[k++] = R[j++];
}

public static void main(String[] args) {
    long programStart = System.nanoTime();

    Scanner sc = new Scanner(System.in);

    System.out.print("Enter array size: ");
    int n = sc.nextInt();

    int[] arr = new int[n];

    System.out.println("Enter array elements separated by space:");
    for (int i = 0; i < n; i++) {
```

```

        arr[i] = sc.nextInt();
    }

    /*Inputs were taken manually from another output of a code.*/

    long sortStart = System.nanoTime();
    mergeSort(arr, 0, n - 1);
    long sortEnd = System.nanoTime();

    System.out.println("Array sorted using Merge Sort:");
    System.out.println(Arrays.toString(arr));

    System.out.println("Merge Sort runtime: " + (sortEnd - sortStart) + "
nanoseconds");

    long programEnd = System.nanoTime();
    System.out.println("Total program runtime: " + (programEnd -
programStart) + " nanoseconds");
}
}

```

4) Quick_Sort :-

```

import java.util.*;

public class QuickSortUser {

    public static void quickSort(int[] arr, int low, int high) {
        if (low < high) {
            int p = partition(arr, low, high);
            quickSort(arr, low, p - 1);
            quickSort(arr, p + 1, high);
        }
    }
}

```

```
public static int partition(int[] arr, int low, int high) {  
    int pivot = arr[high];  
    int i = low - 1;  
  
    for (int j = low; j < high; j++) {  
        if (arr[j] <= pivot) {  
            i++;  
            int temp = arr[i];  
            arr[i] = arr[j];  
            arr[j] = temp;  
        }  
    }  
  
    int temp = arr[i + 1];  
    arr[i + 1] = arr[high];  
    arr[high] = temp;  
  
    return i + 1;  
}  
  
public static void main(String[] args) {  
  
    long programStart = System.nanoTime();  
  
    Scanner sc = new Scanner(System.in);  
    System.out.print("Enter array size: ");  
    int n = sc.nextInt();  
  
    int[] arr = new int[n];  
    System.out.println("Enter array elements separated by space:");  
    for (int i = 0; i < n; i++) {
```

```

        arr[i] = sc.nextInt();
    }

    long sortStart = System.nanoTime();
    quickSort(arr, 0, n - 1);
    long sortEnd = System.nanoTime(); //For measuring the time

    System.out.println("Array sorted using Quick Sort:");
    System.out.println(Arrays.toString(arr));

    System.out.println("Quick Sort runtime: " + (sortEnd - sortStart) + "
nanoseconds");

    long programEnd = System.nanoTime();
    System.out.println("Total program runtime: " + (programEnd -
programStart) + " nanoseconds");
}
}

```

5) Recursive Fibonacci:

```

import java.util.Scanner;

class RecursiveFibonacci {
    static int fib(int n) {
        if (n <= 1)
            return n;
        return fib(n - 1) + fib(n - 2);
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt(); //Inputs were taken manually

        long start = System.nanoTime();
        int result = fib(n);
        long end = System.nanoTime(); //Measuring the time
    }
}

```

```

        System.out.println("Fibonacci(" + n + ") = " + result);
        System.out.println("Time taken: " + (end - start) + " ns");
    }
}

```

6) 0/1 knapsack (Dynamic Programming)

```

import java.util.*;

public class KnapsackRuntimeRandom {

    public static int knapSack(int W, int[] wt, int[] val, int n) {
        int[][] dp = new int[n + 1][W + 1];

        for (int i = 1; i <= n; i++) {
            for (int w = 1; w <= W; w++) {
                dp[i][w] = dp[i - 1][w]; // not taking
                if (wt[i - 1] <= w) { // taking
                    dp[i][w] = Math.max(dp[i][w], dp[i - 1][w - wt[i - 1]] +
val[i - 1]);
                }
            }
        }

        return dp[n][W];
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of items: ");

        int n = sc.nextInt();

        /*Inputs were taken manually from another output of a code.*/
    }
}

```

```

int[] wt = new int[n];
int[] val = new int[n];

Random rand = new Random();

// Generate random weights and values
for (int i = 0; i < n; i++) {
    wt[i] = rand.nextInt(100) + 1; // random weight 1 to 100
    val[i] = rand.nextInt(100) + 1; // random value 1 to 100
}

System.out.print("Enter knapsack capacity: ");
int W = sc.nextInt();

long start = System.nanoTime();
int result = knapSack(W, wt, val, n);
long end = System.nanoTime(); // Measuring the time

System.out.println("\nMaximum value in knapsack = " + result);
System.out.println("Knapsack Runtime (nanoseconds): " + (end - start));
}
}

```

7) Breadth-First Search (BFS):-

```

import java.util.*;

class DenseGraphBFS {

    // BFS function

```

```

static void bfs(ArrayList<ArrayList<int[]>> adj, int src, boolean[] visited)
{
    Queue<Integer> q = new LinkedList<>();
    visited[src] = true;
    q.add(src);

    System.out.println("\nBFS Traversal starting from vertex " + src + ":");
    while (!q.isEmpty()) {
        int u = q.poll();
        System.out.print(u + " "); // print the visited vertex

        for (int[] e : adj.get(u)) {
            int v = e[0];
            if (!visited[v]) {
                visited[v] = true;
                q.add(v);
            }
        }
        System.out.println();
    }
}

// Generate dense graph
static ArrayList<ArrayList<int[]>> generateDenseGraph(int n, int maxWeight)
{
    Random rand = new Random();
    ArrayList<ArrayList<int[]>> adj = new ArrayList<>();
    for (int i = 0; i < n; i++) adj.add(new ArrayList<>());

    for (int u = 0; u < n; u++) {
        for (int v = u + 1; v < n; v++) {
            int w = rand.nextInt(maxWeight) + 1;
            adj.get(u).add(new int[]{v, w});
            adj.get(v).add(new int[]{u, w});
        }
    }
    return adj;
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    System.out.print("Enter number of vertices: ");
    int n = sc.nextInt();

    int maxWeight = 10;

    ArrayList<ArrayList<int[]>> adj = generateDenseGraph(n, maxWeight);
    boolean[] visited = new boolean[n];
}

```

```

    long start = System.nanoTime();
    bfs(adj, 0, visited); // BFS traversal and print
    long end = System.nanoTime(); // Measuring the time

    System.out.println("\nBFS Time (ns): " + (end - start));
    sc.close();
}
}

```

8) Dijkstra's Algorithm:

```

import java.util.*;

public class DijkstraRandom {

    static class Node implements Comparable<Node> {
        int v, w;
        Node(int v, int w) { this.v = v; this.w = w; }
        public int compareTo(Node o) { return this.w - o.w; }
    }

    static ArrayList<ArrayList<Node>> adj;
    static int[] dist;
    static int V;

    public static void dijkstra(int src) {
        PriorityQueue<Node> pq = new PriorityQueue<>();
        boolean[] visited = new boolean[V];

        Arrays.fill(dist, Integer.MAX_VALUE);
        dist[src] = 0;
        pq.add(new Node(src, 0));

        while (!pq.isEmpty()) {
            Node cur = pq.poll();
            int u = cur.v;
            if (visited[u]) continue;
            visited[u] = true;

            for (Node nxt : adj.get(u)) {
                int v = nxt.v;
                int w = nxt.w;

                if (dist[u] + w < dist[v]) {
                    dist[v] = dist[u] + w;
                }
            }
        }
    }
}

```

```

        pq.add(new Node(v, dist[v]));
    }
}

}

}

public static void main(String[] args) {

    Scanner sc = new Scanner(System.in);
    Random rand = new Random();

    System.out.print("Enter number of vertices: ");
    V = sc.nextInt();

    adj = new ArrayList<>();
    dist = new int[V];

    for (int i = 0; i < V; i++)
        adj.add(new ArrayList<>());

    // Generate dense graph
    for (int u = 0; u < V; u++) {
        for (int v = 0; v < V; v++) {
            if (u != v) {
                int w = rand.nextInt(10) + 1;
                adj.get(u).add(new Node(v, w));
            }
        }
    }

    long start = System.nanoTime();
    dijkstra(0);
    long end = System.nanoTime(); //Measuring time

    // Print shortest distances from source
    System.out.println("\nShortest distances from source 0:");
    for (int i = 0; i < V; i++) {
        System.out.println("0 -> " + i + " = " + dist[i]);
    }

    System.out.println("\nTime taken: " + (end - start) + " ns");

    sc.close();
}
}

```

9) Prim's Algorithm:-

```

import java.util.*;

class GFG {

    // Prim's MST function
    static int spanningTree(int V, int edges[][][]) {

        ArrayList<ArrayList<Pair>> adj = new ArrayList<>();
        for (int i = 0; i < V; i++) {
            adj.add(new ArrayList<Pair>());
        }

        for (int i = 0; i < edges.length; i++) {
            int u = edges[i][0];
            int v = edges[i][1];
            int wt = edges[i][2];
            adj.get(u).add(new Pair(v, wt));
            adj.get(v).add(new Pair(u, wt));
        }

        PriorityQueue<Pair> pq = new PriorityQueue<>();
        pq.add(new Pair(0, 0));

        int[] vis = new int[V];
        int mstSum = 0;

        while (!pq.isEmpty()) {
            Pair node = pq.poll();
            int v = node.v;
            int wt = node.wt;

            if (vis[v] == 1)
                continue;

            mstSum += wt;
            vis[v] = 1;

            for (Pair it : adj.get(v)) {
                if (vis[it.v] == 0) {
                    pq.add(new Pair(it.v, it.wt));
                }
            }
        }

        return mstSum;
    }

    public static void main(String[] args) {

```

```

Scanner sc = new Scanner(System.in);
System.out.print("Enter number of vertices (n): ");
int n = sc.nextInt();

Random rand = new Random();

int maxEdges = n * (n - 1) / 2;
int[][] edges = new int[maxEdges][3]; // Generating dense graph
int idx = 0;

for (int i = 0; i < n; i++) {
    for (int j = i + 1; j < n; j++) {
        edges[idx][0] = i;
        edges[idx][1] = j;
        edges[idx][2] = rand.nextInt(1000) + 1;
        idx++;
    }
}

long start = System.nanoTime();
int mstCost = spanningTree(n, edges);
long end = System.nanoTime(); // Measuring the time

System.out.println("\nMinimum Spanning Tree Cost = " + mstCost);
System.out.println("Execution Time (ns): " + (end - start));
}
}

// Pair class
class Pair implements Comparable<Pair> {
    int v;
    int wt;

    Pair(int v, int wt) {
        this.v = v;
        this.wt = wt;
    }

    public int compareTo(Pair that) {
        return this.wt - that.wt;
    }
}

```

10) Floyd-Warshall Algorithm:-

```
import java.util.*;
```

```

class GfG {

    static final int INF = 100000000;

    static void floydWarshall(int[][] dist){
        int V = dist.length;

        for (int k = 0; k < V; k++) {
            for (int i = 0; i < V; i++) {
                for (int j = 0; j < V; j++) {
                    if (dist[i][k] != INF && dist[k][j] != INF)
                        dist[i][j] = Math.min(dist[i][j], dist[i][k] +
dist[k][j]);
                }
            }
        }
    }

    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of vertices (n): ");
        int n = sc.nextInt();

        // create n×n adjacency matrix
        int[][] dist = new int[n][n];
        Random rand = new Random();

        // generate random graph
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                if (i == j) dist[i][j] = 0;
                else {
                    // 50% chance edge exists
                    if (rand.nextBoolean()) {
                        dist[i][j] = rand.nextInt(20) + 1; // weight 1-20
                    } else {
                        dist[i][j] = INF;
                    }
                }
            }
        }

        long startTime = System.nanoTime(); // start

        floydWarshall(dist);

        long endTime = System.nanoTime(); // end
    }
}

```

```

    long duration = endTime - startTime; // Measuring time

    System.out.println("Execution time (ns): " + duration);

    // Print matrix only for small inputs
    if (n <= 10) {
        System.out.println("\nFinal Distance Matrix:");
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                System.out.print(dist[i][j] + " ");
            }
            System.out.println();
        }
    }
}

```

11) TSP (Brute Force):-

```

import java.util.*;

class TSP {

    static final int INF = 100000000;

    // Function to calculate path cost
    static int calculateCost(int[][] dist, int[] path) {
        int cost = 0;
        for (int i = 0; i < path.length - 1; i++) {
            cost += dist[path[i]][path[i+1]];
        }
        cost += dist[path[path.length - 1]][path[0]]; // return to start
        return cost;
    }

    // TSP brute force: try all permutations
    static int tspBruteForce(int[][] dist) {
        int n = dist.length;

        int[] nodes = new int[n];
        for (int i = 0; i < n; i++) nodes[i] = i;

        int minCost = INF;
    }
}

```

```

// Generate all permutations
do {
    int cost = calculateCost(dist, nodes);
    minCost = Math.min(minCost, cost);

} while (nextPermutation(nodes));

return minCost;
}

// Generates next permutation (lexicographically)
static boolean nextPermutation(int[] arr) {
    int i = arr.length - 2;

    while (i >= 0 && arr[i] >= arr[i + 1]) i--;
    if (i < 0) return false;

    int j = arr.length - 1;
    while (arr[j] <= arr[i]) j--;

    int temp = arr[i];
    arr[i] = arr[j];
    arr[j] = temp;

    Arrays.sort(arr, i + 1, arr.length);

    return true;
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    System.out.print("Enter number of cities (n): ");
    int n = sc.nextInt();

    Random rand = new Random();

    int[][] dist = new int[n][n];

    // Generate random symmetric distance matrix
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (i == j) dist[i][j] = 0;
            else dist[i][j] = rand.nextInt(20) + 1; // weights 1-20
        }
    }

    long start = System.nanoTime();

    int result = tspBruteForce(dist);

```

```

    long end = System.nanoTime();
    long duration = end - start; // Measuring the time

    System.out.println("Minimum TSP tour cost = " + result);
    System.out.println("Execution time (ns): " + duration);

    // Print matrix for small n
    if (n <= 10) {
        System.out.println("\nDistance Matrix:");
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                System.out.print(dist[i][j] + " ");
            }
            System.out.println();
        }
    }
}

```

12) TSP(Held-Karp):-

```

import java.util.*;

public class TspHeldKarp {

    private static final int INF = 100000000; // large value for infinity

    // Held-Karp DP algorithm
    public static int solveTSP(int[][] dist) {
        int n = dist.length;
        int VISITED_ALL = (1 << n) - 1;

        int[][] dp = new int[1 << n][n];

        // Initialize DP table
        for (int[] row : dp) {
            Arrays.fill(row, INF);
        }

        // Base case: start from city 0
        dp[1][0] = 0;

        // Fill DP table
        for (int mask = 1; mask < (1 << n); mask++) {
            for (int last = 0; last < n; last++) {

```

```

        if ((mask & (1 << last)) != 0 && dp[mask][last] != INF) {
            for (int next = 0; next < n; next++) {
                if ((mask & (1 << next)) == 0) {
                    int newMask = mask | (1 << next);
                    dp[newMask][next] = Math.min(
                        dp[newMask][next],
                        dp[mask][last] + dist[last][next]
                    );
                }
            }
        }
    }
}

// Complete the tour (return to city 0)
int minCost = INF;
for (int i = 1; i < n; i++) {
    minCost = Math.min(minCost, dp[VISITED_ALL][i] + dist[i][0]);
}

return minCost;
}

// Main method
public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    System.out.print("Enter number of cities (n): ");
    int n = sc.nextInt();

    Random rand = new Random();
    int[][] dist = new int[n][n];

    // Generate random distance matrix
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (i == j) dist[i][j] = 0;
            else dist[i][j] = rand.nextInt(20) + 1; // weights 1-20
        }
    }

    // Start clock
    long startTime = System.nanoTime();

    int result = solveTSP(dist);

    // End clock
    long endTime = System.nanoTime();
    long duration = endTime - startTime; // Measuring time

```

```
System.out.println("Minimum TSP tour cost = " + result);
System.out.println("Execution time (ns) = " + duration);

// Print matrix only for small n
if (n <= 10) {
    System.out.println("\nDistance Matrix:");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            System.out.print(dist[i][j] + " ");
        }
        System.out.println();
    }
}
}
```

...The End...